

Development of a test probe for performing functional tests in OT environments.

Context

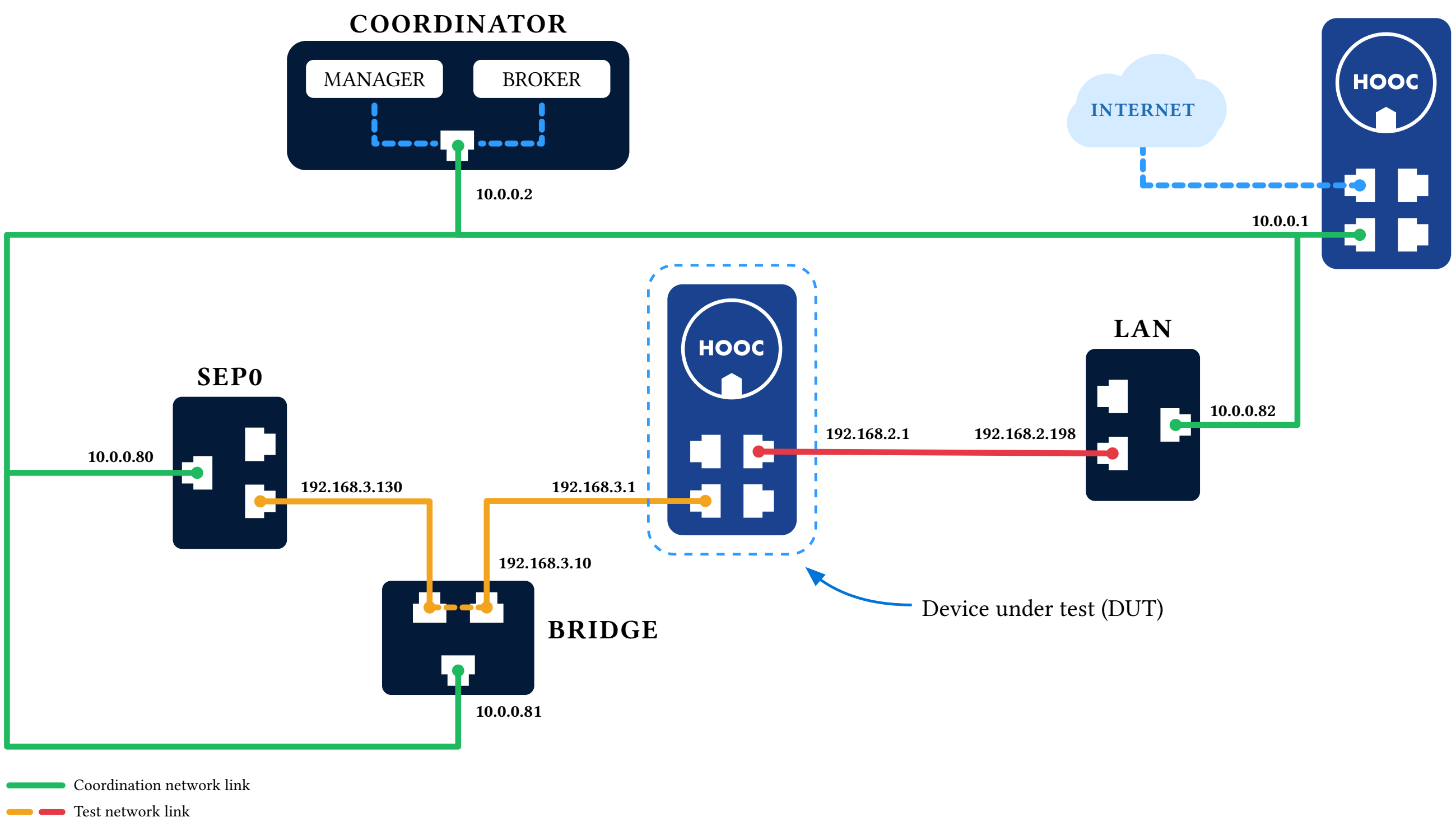
OT environments play a critical role in the industry, as they control and monitor critical processes. Ensuring their proper operation requires functional and security testing. However, implementing these tests often involves manual intervention on multiple network devices, making the process time-consuming, difficult to reproduce, and increases the risk of human error.

The Distributed OT Test Suite addresses this challenge by automating test campaigns through a central coordinator and probes deployed within the OT environment. These probes execute the test commands asked by the coordinator, perform various test operations, and return the results obtained.

Test bench

The test bench was used to validate the probe in a realistic distributed OT environment. It consists of multiple probes, a coordinator, and network devices, allowing the probe's communication, configuration, and testing capabilities to be evaluated.

The architecture of the test bench is illustrated in the diagram below.



Capabilities

- **Ping**
Verify network connectivity and measure the response time.
- **Capture**
Captures the traffic passing through a network interface.
- **Portscan**
Scan a target to identify open ports and available services.
- **HTTP**
Support generic HTTP request command.
- **TCP**
Support sending generic TCP data.
- **UDP**
Support sending generic UDP data.
- **PCAP sending**
Replay predefined network traffic from a PCAP file.

Objectives

The objective of this Bachelor's thesis is to develop a probe that can be deployed within an OT environment to perform distributed test operations.

The probe must fulfill five main objectives.

Register with a coordinator

The probe must register with a coordinator, allowing deployed probes to be identified and managed remotely.

Announce capabilities and configurable parameters

The probe must announce its supported capabilities and configurable parameters, allowing the coordinator to determine which test operations it can perform and which settings can be modified remotely.

Receive and execute commands

The probe must receive and execute commands sent by the coordinator and return the corresponding results.

Support and save configuration

The probe must apply configuration changes received remotely from the coordinator and store them locally so they are preserved across restarts.

Maintain local logs

The probe must maintain local logs of executed commands and configuration changes for monitoring, troubleshooting, and operation tracking.

Architecture

The probe software is organized into several packages, each responsible for a specific part of its functionality. This modular architecture separates the different responsibilities of the application while allowing the components to interact with each other.

The following diagram below presents the packages and their interactions.

